

i3X Strategy for FactoryLM / MIRA

CESMII i3X Alignment -- Research & Recommendation

Generated June 15, 2026

i3X Strategy for FactoryLM / MIRA

Status: Research + strategic recommendation (Phase 0 of the i3X MVP plan)

Date: 2026-06-14

Author: Claude (CHARLIE), research session

Scope: Research-only. No code changes. Companion docs:

`docs/architecture/i3x-aligned-ingestion-and-context-model.md` and

`docs/implementation/i3x-mvp-plan.md`.

*****One-line recommendation:**** Adopt i3X as MIRA's ****outbound read API contract**** — the standard shape through which MIRA exposes its already-contextualized industrial objects and live signals — while keeping MIRA's UNS/KG as the internal master model and ****never**** turning on i3X writes. Position FactoryLM as the *****"industrial context compiler"*****: it ingests messy PLC/SCADA/Ignition/Modbus/OPC UA/MQTT data, digests it into approved contextualized objects + relationships, and serves them through an i3X-conformant surface that any CESMII-ecosystem tool (or LLM via the i3X MCP server) can consume.*

0. How this doc is grounded

Every claim about i3X traces to a fetched primary source; every claim about MIRA traces to a file in this repo. Where a source did **not** confirm something, it is marked **UNKNOWN** rather than inferred — see §9 (validation checklist).

i3X sources fetched (2026-06-14):

Source What it pinned

|---|---|---|

- 1 ``https://www.i3x.dev/`` i3X is "a vendor-agnostic, open, common API"; core capability list (namespaces, object types, instances, relationships, current + historical values, subscribe).
- 2 ``https://github.com/cesmii/API`` (README) "Common API definition for Contextual Manufacturing Information Platforms"; public demo at ``api.i3x.dev/v1``; reference MCP server + Explorer client.

3 ``https://api.i3x.dev/v1/openapi.json`` ****The authoritative endpoint + schema shapes**** (OpenAPI 3.1.0, specVersion ``release``). Full path + schema inventory in §2.

4 ``cesmii/python-i3x-client`` (README) ``i3x.Client`` library; method surface; auth modes (bearer/basic/header); ``verify=`` for self-signed dev certs.

5 ``cesmii/i3X-MCP-Server`` (README) 11 MCP tools mapping i3X → LLM natural-language queries; ``update_value`/`write_history`` ****disabled by default****.

6 ``cesmii/OPCUA-i3X`` (README) OPC UA is an ****upstream adapter****; i3X is the client-facing protocol.

7 ``cesmii/SMProfiles`` (README) SM Profiles = OPC-UA-Information-Model type definitions (nodesets) = the "building blocks of a Namespace"; vendor-neutral; distributed via SM Marketplace.

8 ``https://www.i3x.dev/conformance`` Conformance-suite report labels: ****Full 1.0 Compliance / 1.0 Compatible / Not Compliant****; non-destructive write tests; auth required in production.

9 ``cesmii/i3X`` ``spec/IMPLEMENTATION_GUIDE.md`` (branch ``1.0``) ****The authoritative MUST-vs-MAY endpoint list****, capability flags, elementId/namespace/objecttype rules (§9 checklist).

10 ``cesmii.org/smart-manufacturing-mindset-i3x-explained`` Business framing: "API chaos"; "not about replacing OPC UA or MQTT — about bringing them together."

MIRA sources read: ``mira-relay/tag_ingest.py``, ``mira-relay/clock_resolver.py``
(referenced), ``mira-crawler/ingest/uns.py``, ``mira-bots/shared/uns_resolver.py``,
``mira-mcp/server.py``, ``mira-hub/db/migrations/001_knowledge_graph.sql``,
``.../018_relationship_proposals.sql``, ``.../020_signal_cache_and_trends.sql``,
``.../027_ai_suggestions.sql``, ``.../029_kg_approval_state.sql``, ``.../033_tag_events.sql``,
``.../035_approved_tags.sql``, ``.../036_current_tag_state_freshness.sql``,
``plc/discover.py``, ``docs/specs/uns-kg-unification-spec.md``,
``docs/specs/maintenance-namespace-builder-spec.md``,
``docs/specs/public-ingest-api-spec.md`` §10 (existing ``tol3xEnvelope`` design),
``docs/vision/2026-04-15-mira-manufacturing-gaps.md`` (existing "i3X compliance" intent).

1. What i3X actually is

i3X is an API specification — not a data model, not an ingestion protocol.

- ****What it is:**** an open, vendor-agnostic ****HTTP + JSON read/query API**** (OpenAPI 3.1.0) for accessing **already-contextualized** manufacturing data. It standardizes how an application **reads** objects, types, relationships, current values, and history from any conformant platform — "so apps can be written once and re-deployed against any platform" (paraphrasing the SM Profiles framing).
- ****What it is NOT:****

- ****Not a master data model.**** It does not dictate how you *store* data. It has a *type system* (Namespaces → ObjectTypes with JSON Schema → ObjectInstances → RelationshipTypes), but that is the *exposure* contract, not your internal schema.
- ****Not an ingestion protocol.**** Nothing in i3X tells you how data gets *into* the platform. Ingestion (from PLC/SCADA/OPC UA/MQTT) is entirely the platform's problem. i3X starts where ingestion ends.
- ****Not a replacement for OPC UA / MQTT.**** CESMII is explicit: *"i3X is not about replacing existing technologies like OPC UA or MQTT, it's about bringing them together through a common access platform."* The `OPCUA-i3X`` wrapper treats OPC UA as an **upstream source/adaptor** and i3X as the client-facing target.

The problem it solves (CESMII's framing): "manufacturing data silo proliferation and API chaos." Every historian, MES, CMMS, and analytics platform ships an incompatible API; the industry "risks repeating past fragmentation ... this time at the API layer." i3X is the common access layer that lets one app talk to many platforms.

The mental model: i3X is to contextualized manufacturing data roughly what ODBC/JDBC was to relational databases, or what FHIR is to health records — a **standard query/read façade** over heterogeneous backends. (CESMII itself avoids the ODBC analogy; it's offered here as intuition, not a sourced claim.)

1.1 The i3X type system in one paragraph

A conformant server exposes one or more **Namespaces** (each a unique URI). Each Namespace defines **ObjectTypes** (classes; each MUST carry a **JSON Schema**) and **RelationshipTypes** (each MUST declare a `reverseOf``, and relationships MUST be stored bidirectionally). **ObjectInstances** have a persistent string `elementId``, a `typeElementId`` (pointing at an ObjectType), an optional `parentId`` (which MUST match the `HasParent`` relationship), an `isComposition`` flag, and metadata carrying relationships. Values are **VQT** triples — Value, Quality (`Good | GoodNoData | Bad | Uncertain``), Timestamp (RFC 3339 UTC). History is an ordered array of VQT.

2. The i3X surface (from the live OpenAPI spec)

Pulled from ``api.i3x.dev/v1/openapi.json`` (specVersion ``release``, OpenAPI 3.1.0, server base ``/v1``).

Endpoints

Path Methods Role Conformance

|---|---|---|---|

``/info`` GET server version + capabilities; ****no auth**** ****MUST****, no-auth

``/namespaces`` GET list namespaces ****MUST****

``/objecttypes`` GET list object types (classes) ****MUST****

``/objecttypes/query`` POST query object types ****MUST****

``/relationshiptypes`` GET list relationship types ****MUST****

``/relationshiptypes/query`` POST query relationship types ****MUST****

``/objects`` GET list objects ****MUST****

``/objects/list`` POST fetch objects by elementIds ****MUST****

``/objects/related`` POST graph traversal by relationship ****MUST****

``/objects/value`` POST / ****PUT**** read current value / ****write (MAY)**** read ****MUST****, write ****MAY****

``/objects/history`` POST / ****PUT**** read history / ****write (MAY)**** read ****MUST****, write ****MAY****

``/subscriptions`` POST create subscription ****MUST****

``/subscriptions/register`` ``/unsubscribe`` POST (de)register monitored items ****MUST****

``/subscriptions/sync`` POST polled, acknowledged delivery ****MUST****

``/subscriptions/stream`` POST SSE streaming delivery ****MAY****

``/subscriptions/list`` ``/delete`` POST manage subscriptions ****MUST****

*****Strategic surprise worth flagging:**** a ***read-only*** i3X server is ****not**** just the eight GET-ish endpoints. The base ****subscriptions**** machinery (create / register / sync / list / delete) and ****history read**** are ****MUST****. Only the two ***PUT*** writes and SSE ***stream*** are ****MAY****. So "read-only conformant" still requires implementing polled subscriptions + history. This shapes Phase 2/3 of the MVP plan directly.*

Core schemas (data model)

- ****`Namespace`**** `{ uri, displayName }`
- ****`ObjectTypeResponse`**** `{ elementId, displayName, namespaceUri, sourceTypeId,`

```
version?, schema (JSON Schema), related? }
```

- **ObjectInstanceResponse** { elementId, displayName, typeElementId, parentId?, isComposition, isExtended, metadata? }
- **ObjectInstanceMetadata** { typeNamespaceUri?, sourceTypeId?, description?, relationships?, schemaExtensions?, system? }
- **RelationshipType** { elementId, displayName, namespaceUri, relationshipId, reverseOf }
- **RelatedObjectResult** { sourceRelationship, object }
- **VQT** { value, quality (Good|GoodNoData|Bad|Uncertain), timestamp (RFC 3339 UTC) }
- **CurrentValueResult** { isComposition, value, quality, timestamp, components? }
- **HistoricalValueResult** { isComposition, values: VQT[] }
- **ServerCapabilities** { query, update, subscribe } (each a sub-object of boolean flags: `query.history`, `update.current`, `update.history`, `subscribe.stream`). A server **MUST** return all flags in `/info` regardless of support.

3. Where i3X sits in MIRA's architecture

i3X is a **boundary**, not a core. MIRA already has the layers a contextualized platform needs: i3X is the standard skin over the top of them.

[illegible]

[illegible]

CESMII ecosystem tools · i3X Explorer ·
i3X MCP server · partner analytics/ML ·
MIRA's own LLM (via i3X MCP)

Key placement decisions:

1. **i3X is downstream of the approval gate.** MIRA exposes `*approved context*`, not raw guesses. The ``approved_tags`` allowlist (FAIL-CLOSED) and the KG ``approval_state`` (``proposed`/`verified``, mig 029) are precisely the human gate that decides what becomes an i3X Object. This is a differentiator: most i3X servers expose whatever the historian holds; MIRA exposes only what a technician/admin has confirmed.
2. **UNS + KG stay the master model.** i3X's `ObjectTypes/Instances` are a `*projection*` of MIRA's UNS Itree (``uns.py``) and ``kg_entities`/`kg_relationships``. We do not migrate the internal model to i3X shapes.
3. **OPC UA is an adapter, parallel to Ignition/relay** — never the master. This matches both CESMII's ``OPCUA-i3X`` posture and MIRA's existing ``mira-connect`` (deferred) / ``mira-relay`` separation.

4. Client vs Server vs Both

Recommendation: be a SERVER first, a CLIENT opportunistically, and a write-server never (in MVP).

Role Verdict Rationale

|---|---|---|

****i3X read server**** ****YES — primary**** This is the product wedge. MIRA already holds contextualized objects + live values; exposing them as i3X makes MIRA a citizen of the CESMII ecosystem and a drop-in backend for any i3X app. Directly realizes the existing "I3X API server exposes only read operations externally" intent in ``docs/vision/2026-04-15-mira-manufacturing-gaps.md``.

i3X client ****YES** — later / opportunistic** Where a customer **already** runs an i3X-conformant platform (or the ``OPCUA-i3X`` wrapper), MIRA can ****ingest via i3X**** using ``cesmii/python-i3x-client`` instead of writing a bespoke adapter. This makes i3X an **ingestion adapter option**, not the master. Lower priority than server because few sites have i3X servers today.

****i3X write server**** (`PUT /objects/value`, `PUT /objects/history`) ****NO** — explicitly declare unsupported**
Writes to the plant are out of MIRA's scope (`fieldbus-readonly.md`, SaaS scope guard). MIRA's `/info` **MUST**

return `capabilities.update.current=false` and `capabilities.update.history=false`. This is **fully conformant** — writes are MAY, not MUST.

****i3X MCP server**** ****YES** — reuse CESMII's ``cesmii/i3X-MCP-Server`` already maps i3X → 11 LLM tools. Once MIRA serves i3X, MIRA's own LLM (and customers' LLMs) get plant access "for free" via that MCP server, with writes disabled by default. See §6.

5. Business positioning — the "industrial context compiler"

A compiler takes messy, source-language input and emits a clean, typed, machine-consumable artifact in a standard format. That is exactly the FactoryLM pitch, and i3X is the standard output format that makes the metaphor literal.

- ****Input (source languages):**** PLC registers, SCADA tags, Ignition tag exports, Modbus maps, OPC UA address spaces, MQTT/Sparkplug B topics — none of which agree on naming, semantics, or quality.
- ****Compilation passes (MIRA's value-add):**** normalize → classify (command / feedback / status / fault / alarm / analog / speed / current ...) → attach to an asset → relate (HasComponent / ControlledBy / Drives / Monitors / AlarmFor ...) → **gate through human approval** → store as typed objects in the UNS/KG.
- ****Output (target language):**** i3X-conformant objects, types, relationships, VQT values, and history — the **linkable, queryable, LLM-ready** artifact.

Why this positioning wins:

1. **It is honest about the hard part.** Competitors sell "connect your PLC." FactoryLM sells "we turn the connection into **meaning** you can trust" — the classification + relationship + approval passes are the moat, and they are exactly what raw i3X servers (historian-backed) lack.
2. **i3X is the distribution channel, not the product.** Conforming to i3X means every CESMII-ecosystem buyer can adopt MIRA without lock-in fear — the same "no lock-in" promise that sells i3X sells MIRA **through** i3X.
3. **It compounds with the SM Profile / template flywheel.** A vendor-neutral ``ConveyorDrive`` ObjectType (cf. SM Profiles, the existing template library in ``maintenance-namespace-builder-spec.md``) compiled once and confirmed at Site A auto-applies at Site B. i3X is the wire format that lets those compiled profiles travel (potentially via the CESMII SM Marketplace).

The tagline: ***FactoryLM is the industrial context compiler: it ingests every*

messy plant signal and emits trustworthy, i3X-standard industrial objects an AI can reason over without hallucinating."

6. How i3X reduces MIRA hallucination

Hallucination in industrial diagnosis comes from the LLM inventing plant context (tag meaning, asset structure, fault semantics). MIRA's whole doctrine is **grounding** (the UNS gate, citation compliance, groundedness scoring). i3X sharpens that in four concrete ways:

- 1. Typed objects instead of free-text.** An i3X `ObjectInstance` has a ``typeElementId`` → an `ObjectType` with a JSON Schema. The LLM no longer guesses "what is ``HR400110``?" it reads a typed ``DCBusVoltage`` analog object with units and bounds. Schema-typed context is far harder to hallucinate over than a raw tag string.
- 2. VQT quality + timestamp on every value.** ``quality`` ∈ {Good, GoodNoData, Bad, Uncertain} + RFC 3339 timestamp means the model is *told* when data is stale/bad and must say so — instead of confidently reading a dead sensor. MIRA already carries ``latest_quality`` + ``freshness_status`` (mig 036) + clock provenance (``clock_resolver``); i3X gives that a standard, model-legible shape.
- 3. Relationships make "what's connected" answerable, not inferable.** With ``/objects/related`` over ``HasComponent``/``ControlledBy``/``Drives``/``Monitors``/``AlarmFor``, the LLM *traverses* the real graph instead of pattern-matching a plausible-sounding topology. This is the same anti-hallucination lever the MCP ``feeds``/``caused_by`` traversal tools already provide internally — i3X standardizes it for any consumer.
- 4. The approval gate guarantees the model only sees confirmed context.** Because i3X sits *downstream* of ``approved_tags`` + KG ``approval_state``, anything the LLM reads via i3X is human-verified. ``proposed`` (unconfirmed) entities are not exposed. The model literally cannot ground on a guess, because guesses never reach the i3X surface.

*Net: i3X turns MIRA's existing groundedness machinery into a *standard, queryable, scoped* context source — which is exactly what the manufacturing-gaps*

doc identified ("scopes context to the specific equipment object being worked on ... the agent always knows the schema of what it's operating on").

7. How i3X enables MCP tools

This is the highest-leverage, lowest-effort win.

- **CESMII already built the bridge.** ``cesmii/i3X-MCP-Server`` exposes 11 MCP tools over any i3X server: ``connect``, ``connection_status``, ``server_info``, ``search_objects`` ("the main 'where is X' tool"), ``list_root_objects``, ``refresh_catalog``, ``get_object``, ``read_current_value``, ``get_history``, ``find_related``, ``describe_type``, ``watch_values`` — with ``update_value`` / ``write_history`` **disabled by default**.
 - **Once MIRA serves i3X, MIRA gets a plant-aware MCP surface for free.** A technician's LLM (or MIRA's own engine) can ask "where is the conveyor 2 drive, what's its DC bus voltage, and what alarms relate to it?" and get grounded, typed answers — without MIRA writing a single new MCP tool.
 - **It complements (does not replace) ``mira-mcp/server.py``.** MIRA's internal MCP server is diagnosis/CMMS/UNS-specific and stays. The i3X MCP server is the **standard, externally-shaped** read surface. Two MCP servers, two audiences (internal diagnosis vs. ecosystem read) — consistent with the public-ingest spec's recommendation to keep the public MCP surface decoupled.
 - **Writes-disabled-by-default aligns with MIRA doctrine.** The i3X MCP server's default matches ``fieldbus-readonly.md`` and the SaaS scope guard exactly.
-

8. Relationship to MIRA's **existing** i3X intent (do not fork)

MIRA already has i3X intent on ``origin/main``. This strategy **extends and supersedes** it into one coherent contract — it does not introduce a second.

- ``docs/specs/public-ingest-api-spec.md`` §10 defines a forward-compatible ``toI3xEnvelope(row, kind)`` helper (planned at ``mira-hub/src/lib/i3x.ts``, **not yet built**) and a small mapping table (Asset→Object, Component→Object w/ ``HasComponent``, tag value→VQT, etc.). **Decision:** keep that mapping as the seed; this doc's §2/§3 + the architecture doc's layer model are the

authoritative, expanded version.** When `i3x.ts` is built, it implements
this mapping, including the `elementId` rule in §8.1. The public-ingest spec's
§10 should be updated to point here (a one-line cross-reference), so there is
exactly one i3X mapping in the repo.

- `docs/vision/2026-04-15-mira-manufacturing-gaps.md` already asserts an "I3X compliance spec," a read-only external i3X server, and SM-Profile-as-ground-truth. This doc is the concrete realization of that vision.
- `docs/specs/dt-scorecard-spec.md` + `factorylm.com/assess` is a CESMII Smart Manufacturing maturity scorecard — adjacent CESMII alignment, not i3X itself.

8.1 The `elementId` decision (resolves an open ambiguity)

i3X requires `elementId` to be **unique** and **SHOULD be persistent**. MIRA has two candidate identifiers: the UNS ltree path and the `kg_entities.id` UUID. **Use the stable UUID as `elementId`; carry the UNS path as `displayName`/metadata.**

Rationale: UNS paths *mutate* on site reassignment (a kb model relinked to a site via `INSTANCE_OF` — see `uns.py`), which would violate persistence and break relational references (`parentId`, `typeElementId`). UUIDs never move. This also matches the existing §10 choice (`ElementId=asset.id`). The UNS path remains the human-readable address and the namespace organizer.

9. Validation checklist (PASS / FAIL / UNKNOWN)

Each row is a claim this research needed to settle. **PASS** = confirmed by a fetched/inspected source. **FAIL** = confirmed false / a real gap. **UNKNOWN** = not confirmed by any source consulted (→ Phase 0 follow-up).

About i3X itself

Claim Verdict Evidence

|---|---|---|---|

- 1 i3X is an API spec, not a data-model master or ingestion protocol ****PASS**** i3x.dev, cesmii.org framing; OpenAPI defines access only
- 2 i3X does not replace OPC UA / MQTT ****PASS**** cesmii.org: "not about replacing ... bringing them together"
- 3 i3X models objects, types, relationships, current value, history, subscriptions ****PASS**** OpenAPI paths + schemas

- 4 Reads are MUST; writes (`PUT`) are MAY/optional ****PASS**** IMPLEMENTATION_GUIDE: PUT value/history are "Update Methods (MAY)"
- 5 Base subscriptions + sync + history-read are MUST (not optional) ****PASS**** IMPLEMENTATION_GUIDE MUST list
- 6 SSE `/subscriptions/stream` is MAY ****PASS**** IMPLEMENTATION_GUIDE; gated by `subscribe.stream` flag
- 7 `/info` MUST NOT require auth; all else MUST require auth in production ****PASS**** IMPLEMENTATION_GUIDE
- 8 Auth scheme is not mandated ****PASS**** IMPLEMENTATION_GUIDE + python client (bearer/basic/header)
- 9 Every ObjectType MUST have a JSON Schema; MUST belong to exactly one Namespace ****PASS**** IMPLEMENTATION_GUIDE
- 10 Every RelationshipType MUST define `reverseOf`; relationships stored bidirectionally ****PASS**** IMPLEMENTATION_GUIDE
- 11 `elementId` MUST be unique, SHOULD be persistent + human-readable ****PASS**** IMPLEMENTATION_GUIDE
- 12 VQT quality enum = {Good, GoodNoData, Bad, Uncertain} ****PASS**** OpenAPI `VQT.quality` description
- 13 OPC UA is treated as an adapter, not the master model ****PASS**** OPCUA-i3X README
- 14 A reference i3X→MCP server exists (writes off by default) ****PASS**** i3X-MCP-Server README (11 tools)
- 15 A pip-installable Python i3X client exists ****PASS**** python-i3x-client README (`pip install i3x-client`)
- 16 SM Profiles = OPC-UA-model type definitions = Namespace building blocks ****PASS**** SMProfiles README
- 17 Formal named conformance *tiers* ("Full 1.0 / 1.0 Compatible") are defined in the spec ****FAIL (nuance)**** IMPLEMENTATION_GUIDE defines no formal tiers; the ****conformance suite**** *reports* those labels (i3x.dev/conformance). Conformance = implement all MUSTs + accurately report capabilities.
- 18 Exact structural OPC UA node/reference → i3X object/relationship mapping ****UNKNOWN**** OPCUA-i3X README explicitly did not describe it
- 19 Exact SM Profile → i3X ObjectType binding mechanics (nodeset → JSON Schema) ****UNKNOWN**** Neither SMProfiles nor i3x.dev detailed the binding; SM Profiles "Part 2" mentions W3C WoT JSON as a *candidate*
- 20 i3X spec version is finalized at 1.0 ****PASS**** OpenAPI specVersion `release`; cesmii API README cites v1.0 finalized June 2026

About MIRA's readiness to serve i3X

Claim Verdict Evidence

|---|---|---|---|

- 21 MIRA has a raw, append-only capture layer (→ history) ****PASS**** `tag\_events` mig 033; `tag\_ingest.py`
- 22 MIRA has a current-value layer (→ `/objects/value`) ****PASS**** `live\_signal\_cache` mig 020/036 (= "current_tag_state")

23 MIRA has a human approval gate before exposure **PASS** `approved_tags` FAIL-CLOSED (035); KG approval_state` proposed/verified (029); ai_suggestions` (027)`

24 MIRA has a typed object/relationship store ($\rightarrow$ objects + relationships) **PARTIAL/PASS** `kg_entities`/kg_relationships` (001) exist, but types are bare TEXT — see #27`

25 MIRA has a quality + freshness concept ($\rightarrow$ VQT quality) **PASS** `latest_quality` + `freshness_status` (036); not 1:1 with i3X enum — see #28`

26 MIRA has a relationship-traversal capability ($\rightarrow$ `/objects/related`)` **PASS** MCP `feeds`/caused_by`/sequence-walk tools in `mira-mcp/server.py``

27 MIRA emits a JSON Schema per object type (i3X ObjectType requirement) **FAIL (gap)** `kg_entities.entity_type` is bare TEXT; no per-type JSON Schema, no Namespace URI`

28 MIRA's quality codes map 1:1 to i3X VQT quality **FAIL (gap)** MIRA {good,bad,stale,uncertain}; i3X {Good,GoodNoData,Bad,Uncertain}. `stale` $\rightarrow$ Uncertain (+ freshness); GoodNoData` unmodeled`

29 MIRA's RelationshipTypes have reverse pairs (i3X requirement) **FAIL (gap)** `relationship_type` is a single TEXT direction; no reverseOf` vocabulary, not stored bidirectionally`

30 MIRA has Namespace URIs for its type vocabulary (i3X requirement) **FAIL (gap)** UNS addresses `*instances*`; there is no typed-namespace URI for the `*type*` vocabulary

31 MIRA has any OPC UA integration today **FAIL (absent)** `mira-connect` is deferred; no asynqua`/OPC UA client in tree. OPC UA ingestion is greenfield (or via OPCUA-i3X`)`

32 MIRA already wrote `mira-hub/src/lib/i3x.ts`` **FAIL (not built)** Planned in public-ingest §10; file does not exist

33 MIRA's `kg_relationships` approval lives in migration 001` **FAIL (correction)** 001 has no approval column; `approval_state` added in mig 029`

The five FAIL-gap rows (#27–#30) are the technical heart of the work and drive

the architecture doc's gap analysis: i3X wants a `*typed*` model (namespace URIs + JSON-Schema'd ObjectTypes + reverse-paired relationships), and MIRA's KG types are bare strings today.

10. Strategic recommendation (summary)

- Adopt i3X as the outbound read contract.** Build a read-only, conformant-by-capability i3X server as a projection of UNS + KG + `live_signal_cache` + tag_events`. Declare update.*` capabilities false`.`
- Position FactoryLM as the industrial context compiler** — the ingestion + classification + relationship + approval passes are the product; i3X is the standard output and distribution channel.
- Reuse CESMII's MCP server + Python client** — don't rebuild. The MCP server gives MIRA a grounded, plant-aware LLM surface for free; the Python client is

the path to *ingesting* from sites that already run i3X.

4. **Treat OPC UA (and everything else) as an adapter** feeding raw capture; the UNS/KG stays the master model.

5. **Close the typed-model gaps (#27–#30)** as the real engineering: per-type JSON Schemas, Namespace URIs, reverse-paired relationship vocabulary, and a VQT quality mapping. These are additive to the KG, not a rewrite.

6. **Keep one i3X mapping in the repo** — extend public-ingest §10, don't fork.

7. **Hold the read-only / no-historian-required / no-Ignition-8.3-required line** throughout (see the MVP plan's "what NOT to do" sections).

Next: ``docs/architecture/i3x-aligned-ingestion-and-context-model.md`` (the layer-by-layer technical model) and ``docs/implementation/i3x-mvp-plan.md`` (the phased build).